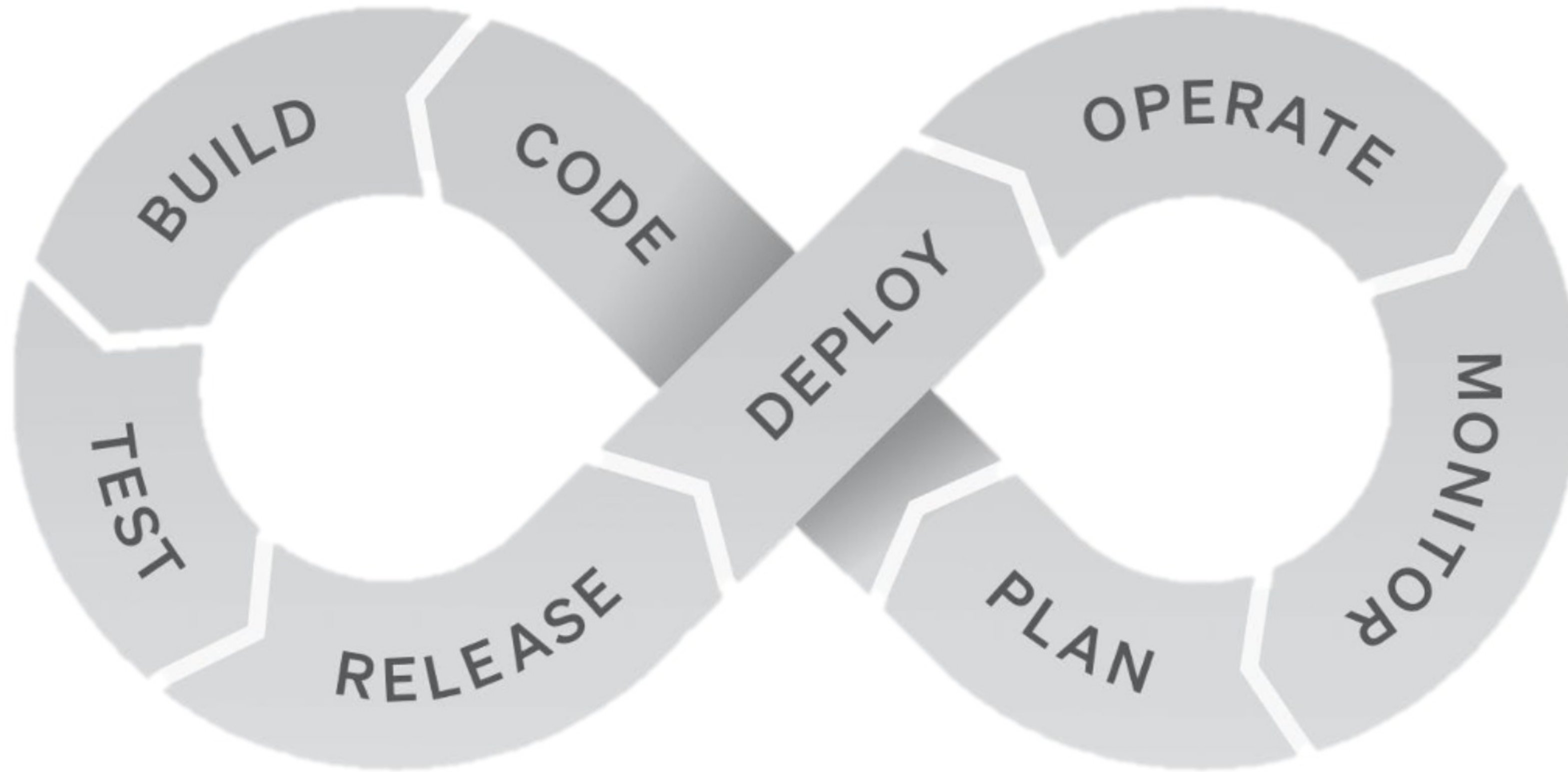


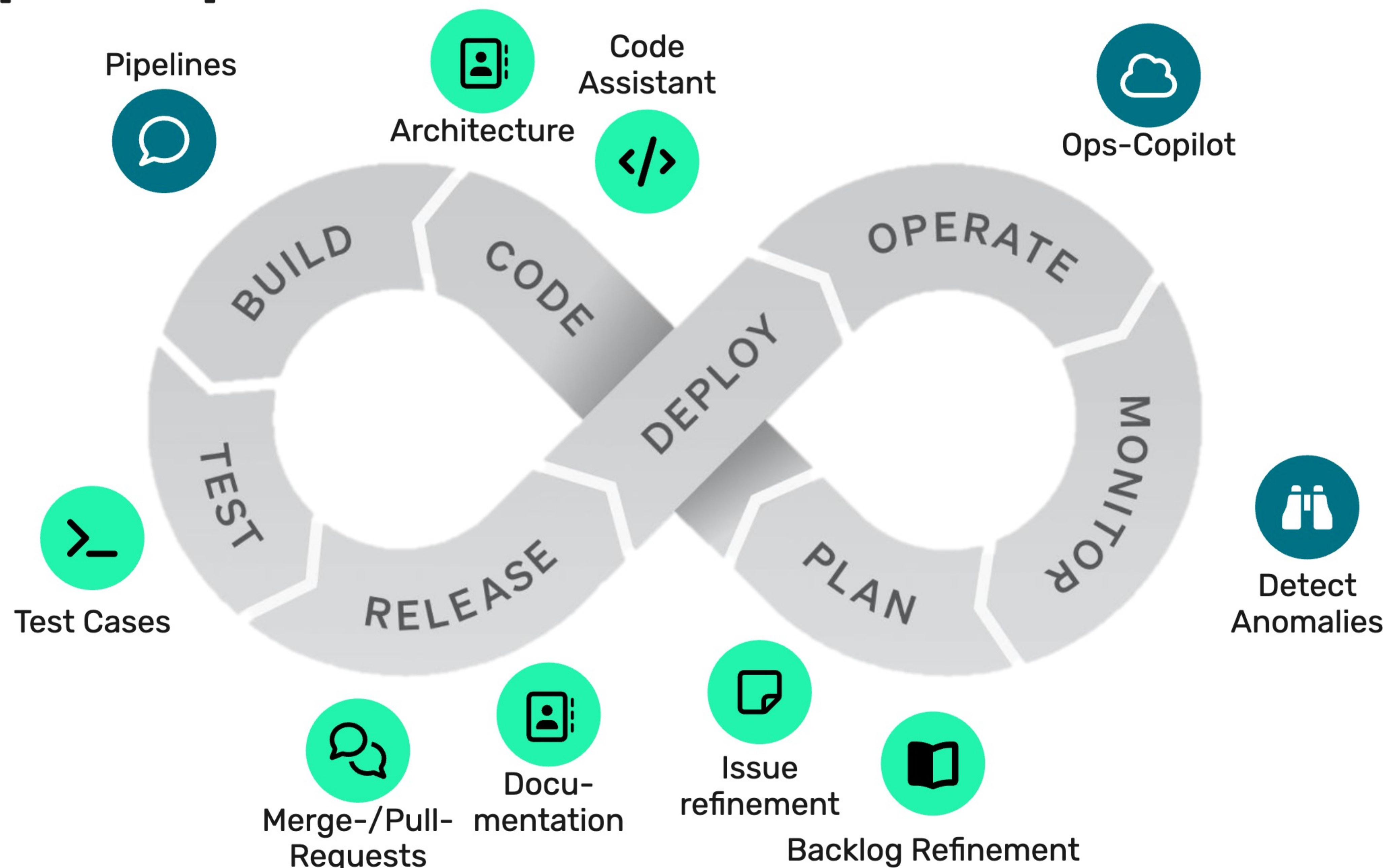


**Use AI in the
whole
Software
Development
Lifecycle
(SDLC)**

DevOps-Loop



DevOps-Loop



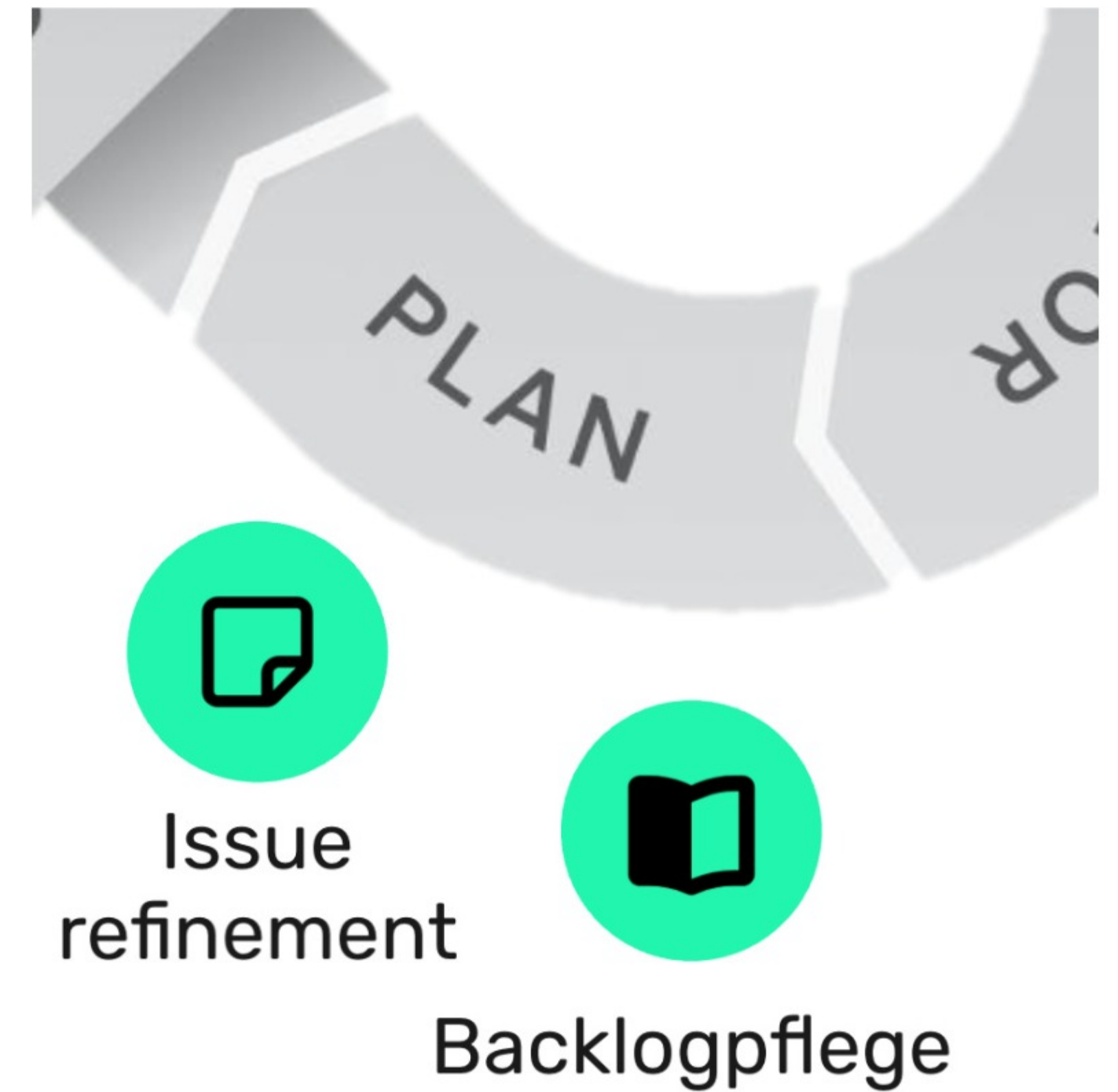
01

Story Refinement

AI for Backlog and Stories

AI as a Sparring Partner for

- Product Vision
- Backlog Maintenance
- Writing User Stories
- Refinements



Definition of Ready

"We need a 'definition of ready' for our project, so that each issue has a defined quality.

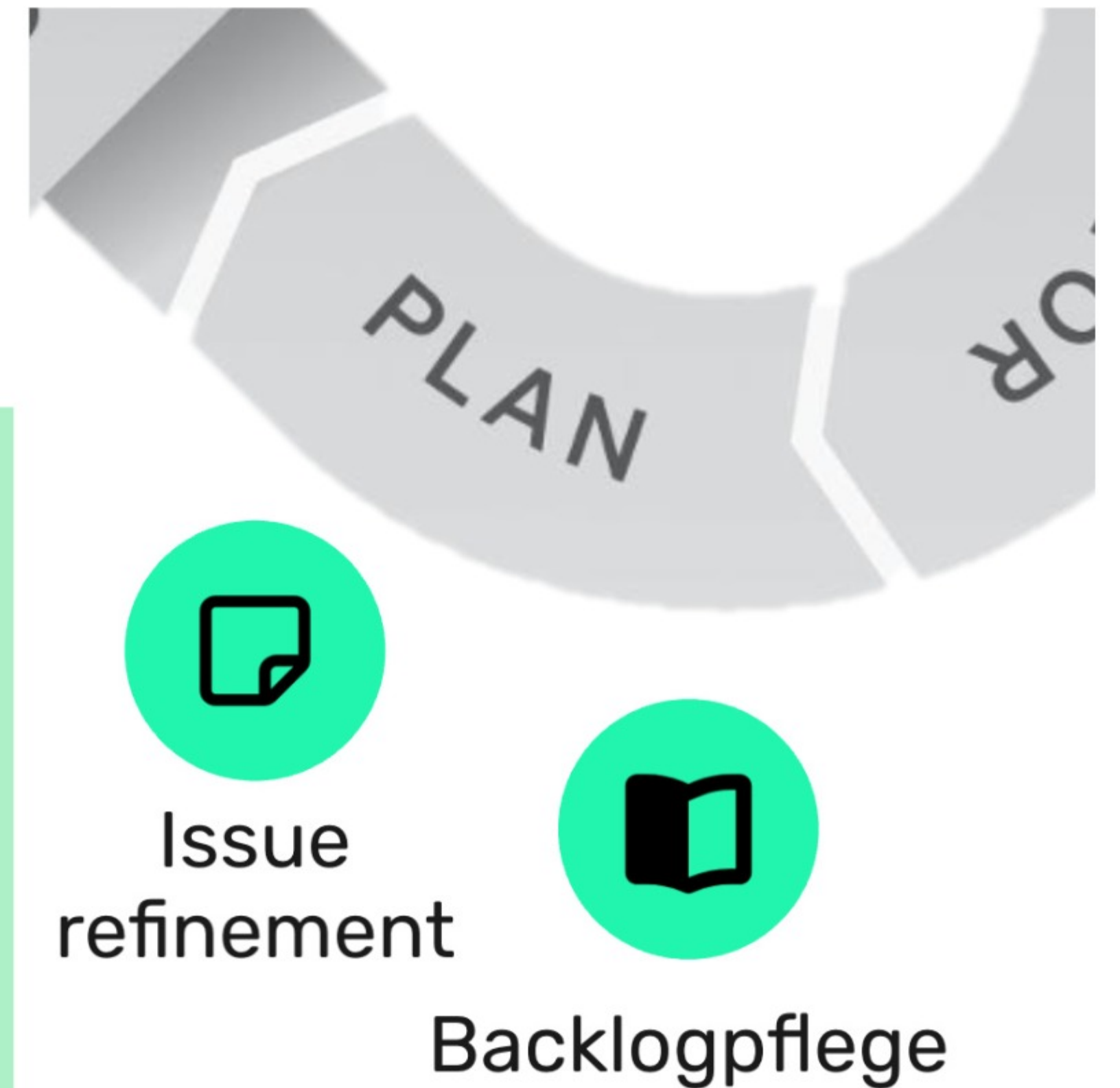
It should include

- story sentence ('as a ... I want to ... so that ...')
- business details if needed
- acceptance criteria

What other aspects should be included in a story?

Keep it brief and concise.

Save the definition as docs/definition_of_ready.md"

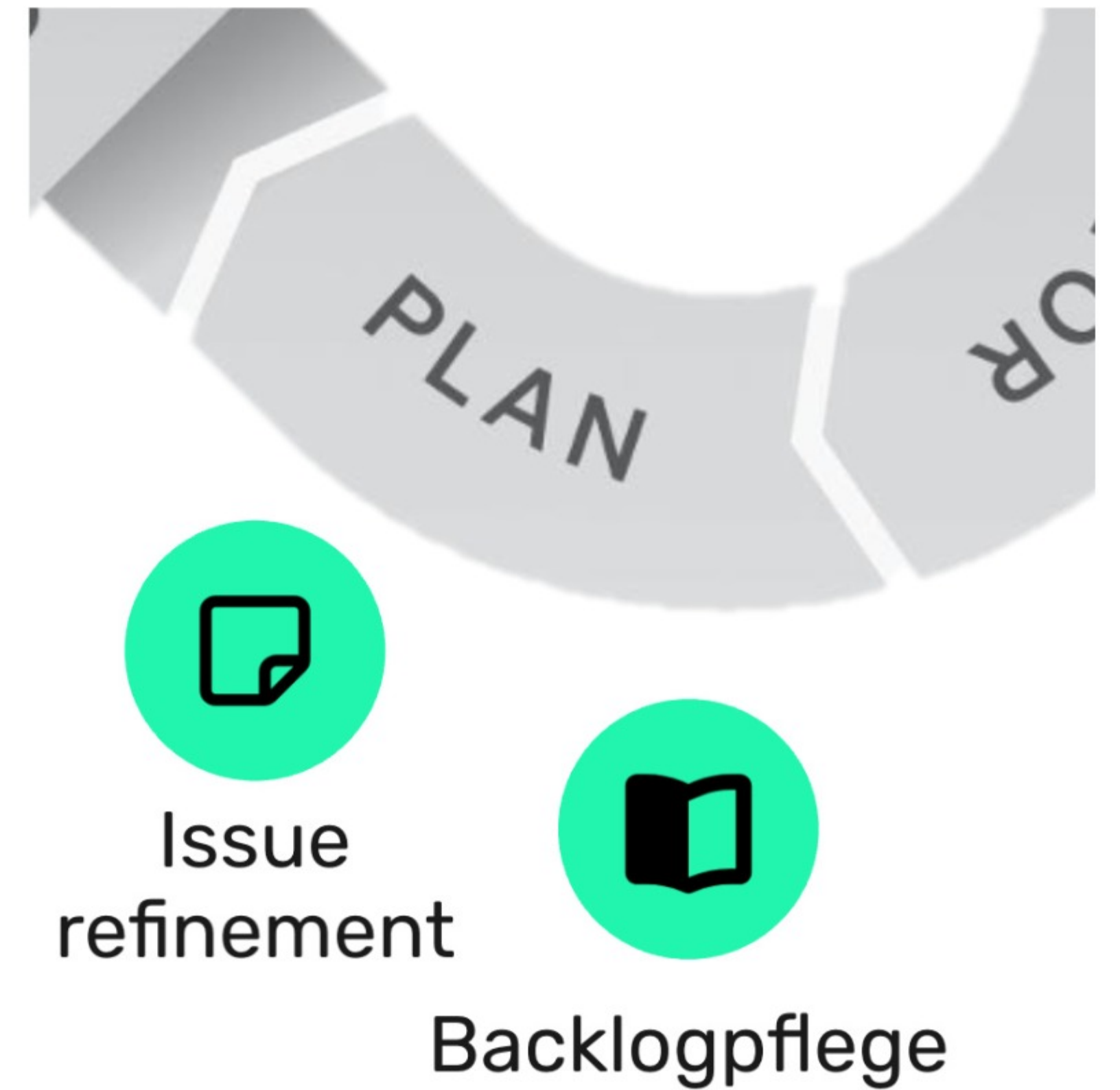


Story refinement

"Take a look at the following story

As a ... I want to ... so that ...

Refine this story according to
[@docs/definition_of_ready.md](#)"



Task: Story Refinement



Phase 1 Plenum

Definition of Ready

1. As a Team, discuss with the AI about a Definition of Ready
2. Store the file in markdown format
3. Share with all team member

10min

Phase 2 Plenum

Explore and Select

1. Look at your backlog
 2. Discuss and Select 1 or more stories to refine
- (not too large, not too small, not too much Detail)

5min

Phase 3

Refinement

1. Work alone or in Pairs to refine the story
2. Make sure, it meets the "Definition of Ready"

20min

Phase 4 Plenum

Share

- Closing review with all members

10 min

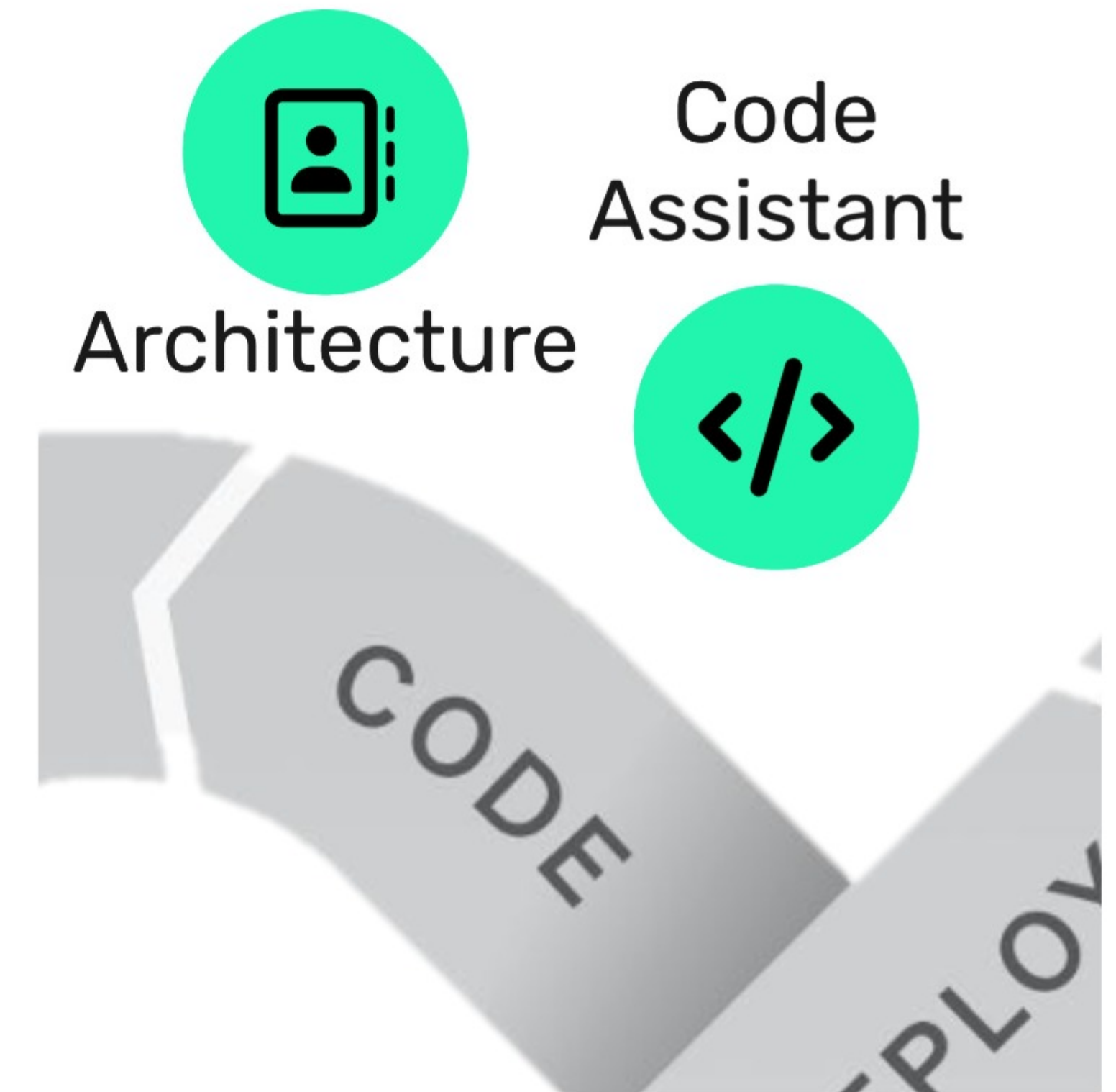
02

Planning & Implementation

AI in Coding

Use AI to support software development

- Architecture documentation
- Code style
- Test coverage
- Specification
- Implementation



How To Think Of An AI Agent

You have to treat a coding AI agent just like

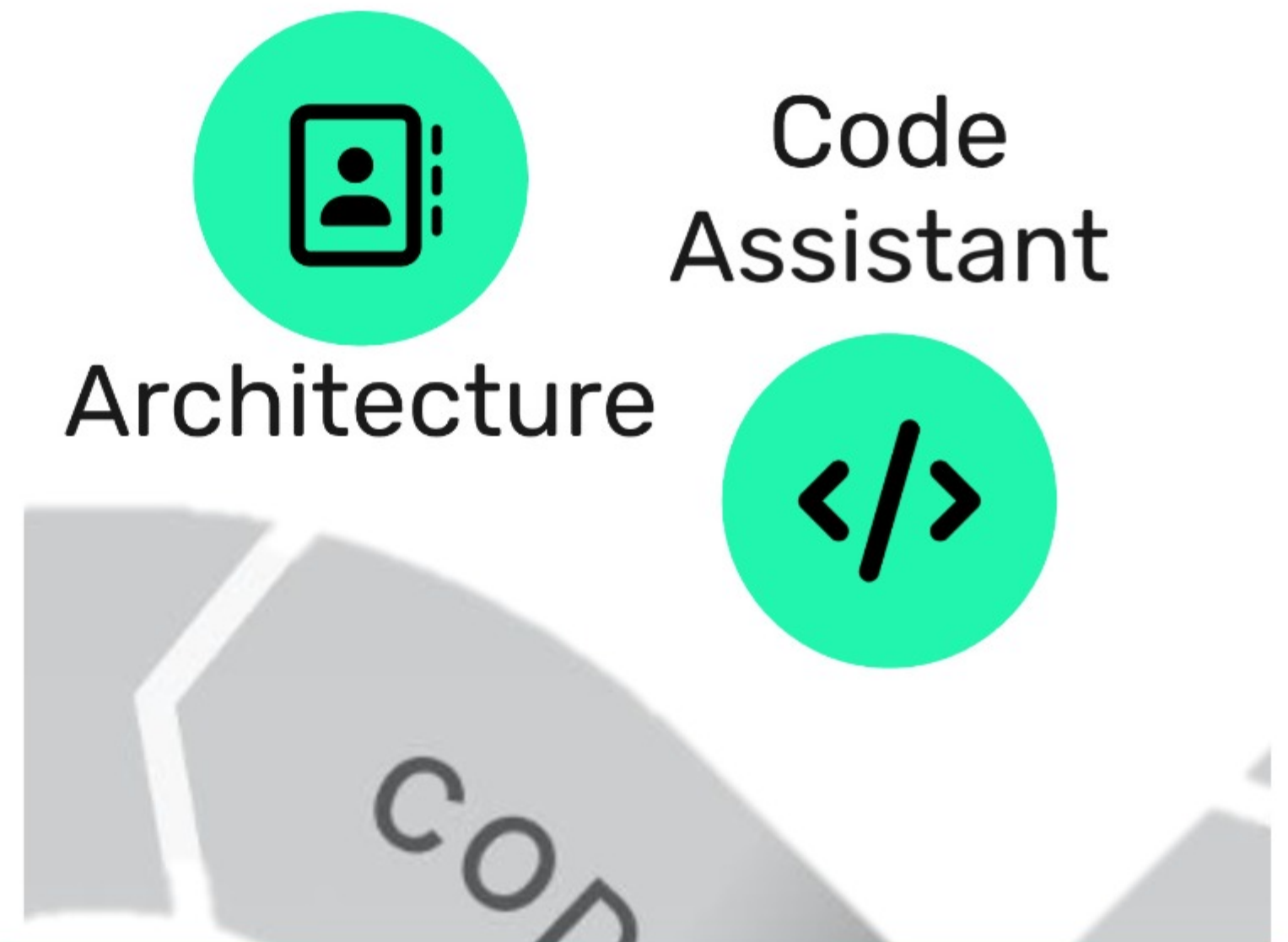
- a senior developer
- woken up at 3 am
- sitting in front of a completely unfamiliar project
- but: able to read extremely quickly

What sort of information would you want to give someone like that?



Spec Driven Development

Using AI agents in a targeted and controlled manner, for example:



Plan

Story refinement and the development of a detailed implementation plan

Execute

Implement the story in accordance with the plan and the architecture.md / code_style.md ...

Review

Check that implementation follows the plan, architecture.md, definition_of_done.md etc., and draw up a report



Plan

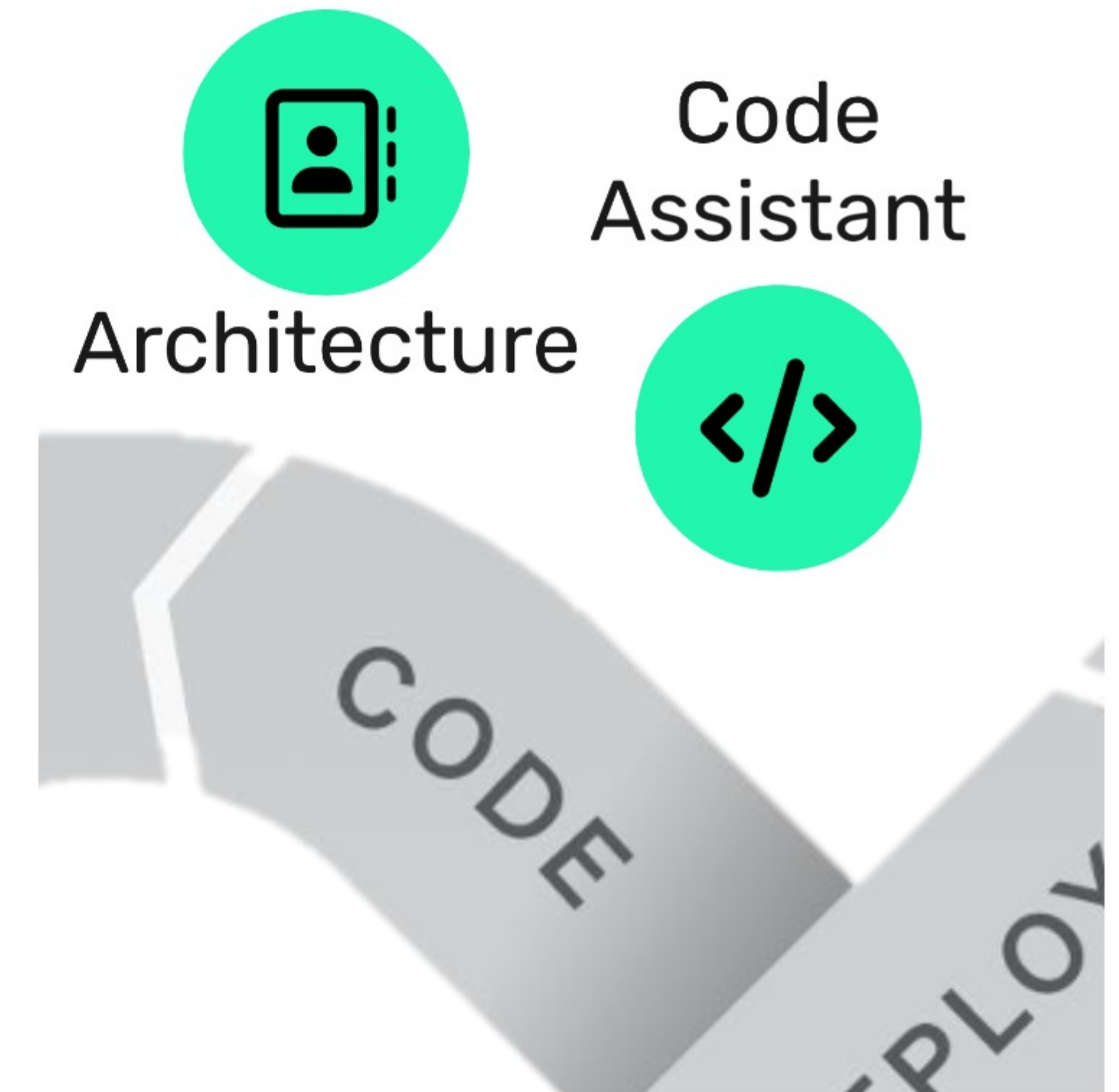
Create an *implementation plan* for story ``story_x_refined.md``. Explore the current implementation.

What *parts* of the application must be changed and why?

Which *tests* should be written?

Follow ``architecture.md`` and ``code_style.md``!

Save the plan as ``plan_x.md``



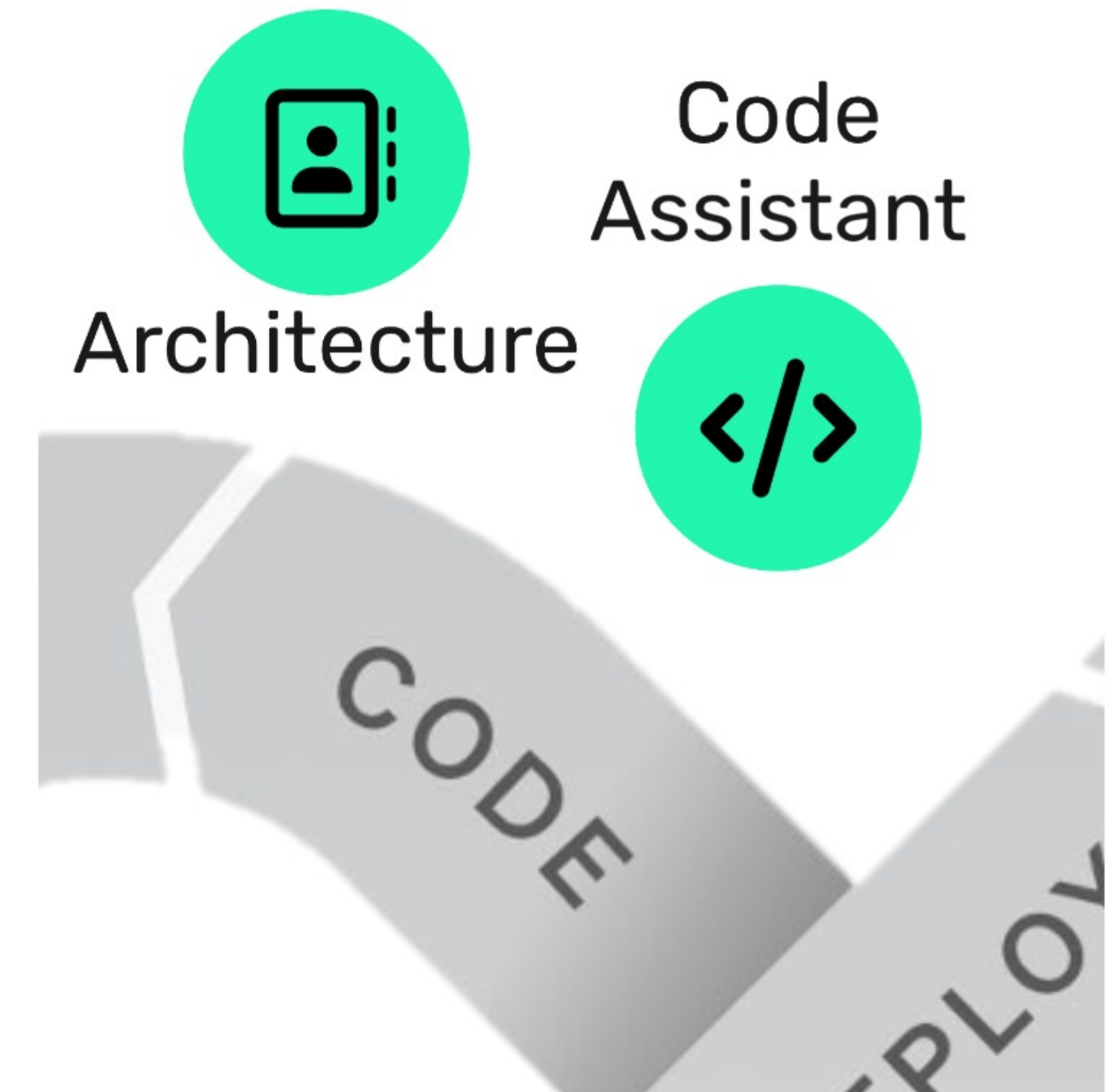
Plan 2

A senior developer has created a plan for
`story\_x\_refined.md` in `plan\_x.md`.

Review this plan.

What are its **strengths** and what its **weaknesses**?
What would you **change and why**?

Write an **updated version** of the plan in
`plan\_x\_v2.md`

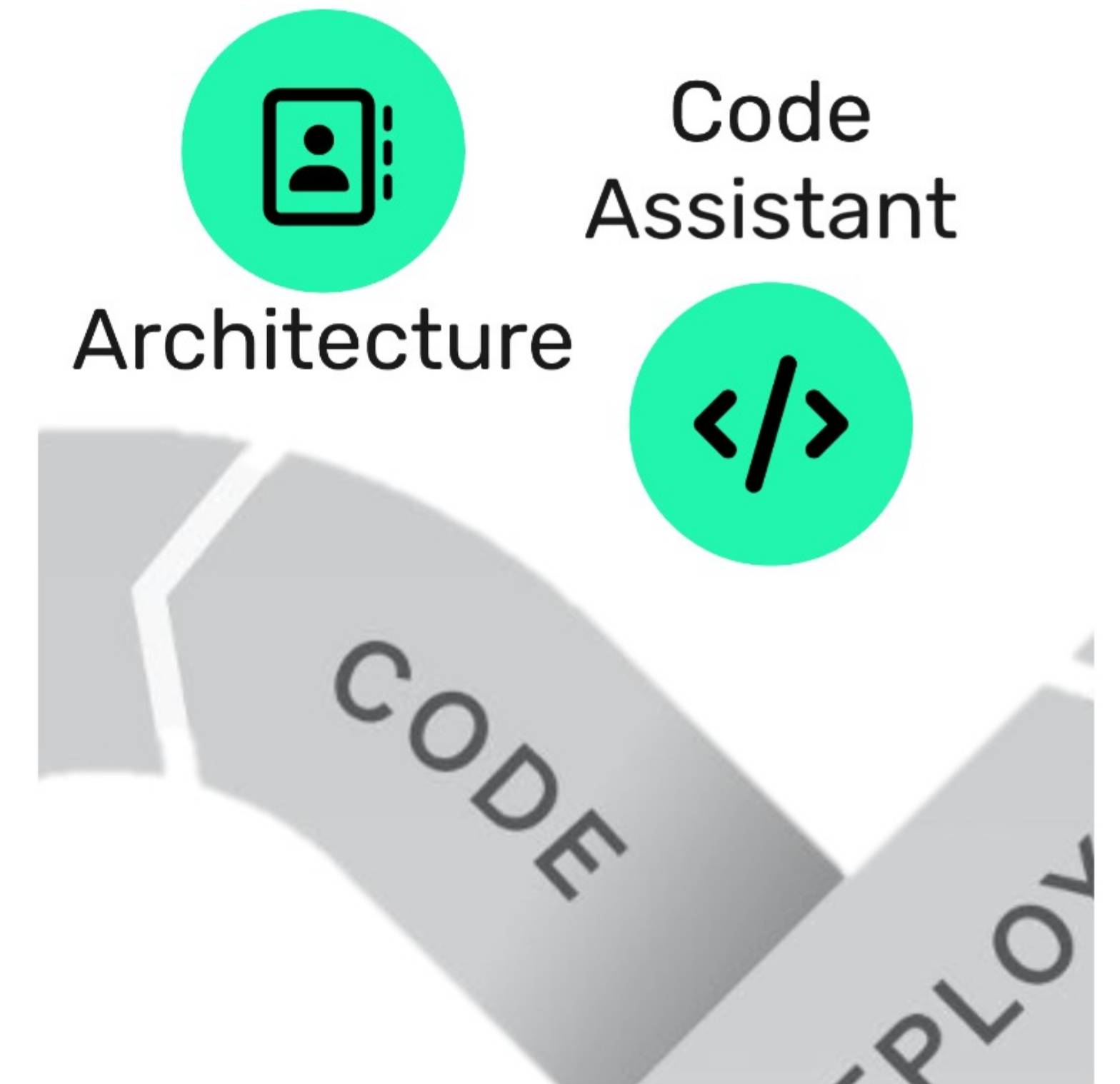


Execute

Work on ``story_x_refined.md`` according to the plan in ``plan_x_v2.md``.

If it makes sense, use sub agents for the individual tasks to reduce token usage on main session.

Follow ``architecture.md`` and ``code_style.md``!



Task: Planning & Implementation



Use AI to bring the first story to life

Phase 1

Planning

1. Take the **refined story** from the last task
 2. If not done, create a **`architecture.md`** file
 3. Create an **implementation plan** for the story
 4. Let **AI review** the plan
 5. Check the plan **manually**
- 20min

Phase 2

Execute

1. Let the AI **execute the plan**
- 10min

Phase 3 Plenum

Share

- Share your results and experience with the group

03

Review & Rework

Test Driven Development

Tests help to verify that an AI output achieves its objective and does not cause any side effects in other areas.

Tests must **always** be checked manually.

Subtle bugs in tests --> Tests are **only** valuable if you've actually seen them fail!



Testfälle



Definition of Done

"We need a 'definition of done' for our project, so that each implementation has a defined quality and follows the same rules.

It should include

- readability
- maintainability
- test coverage
- following ``architecture.md`` and ``code_style.md``
- acceptance criteria

What other aspects should be included in this definition?

Save the definition as `docs/definition_of_done.md`



Testfälle



Review

Immer in einer separaten Session

Review the current changes that were made for
`story\_x\_refined.md` according to the plan in
`plan\_x\_v2.md`.

Is the implementation **consistent**, **secure**,
maintainable, ...?

Does it follow the `architecture.md`, the
`code\_style.md` and the `definition\_of\_done.md`?

Save the review as `review\_x.md`



Merge-/ Pull-
Requests



Rework

Immer in einer separaten Session

Take a look at `review\_x.md`. Check all detected bugs or places for rework.

Work on these issues one by one.

Follow `architecture.md` and `code\_style.md`!



Merge-/ Pull-
Requests

Task: Review & Rework



Phase 1

Definition of Done

1. Create a ``definition_done.md`` file with the help of AI
2. Let AI review the file
3. Check the file manually and enhance it

Stay under 200 lines
10min

Phase 2

Review & Rework

1. Let AI check the code according to the ``definition_done``
2. Check the findings manually
3. Select which finding is worth the rework
4. Let AI work on some/all of the finding
5. Let AI review the code again

Repeat until satisfied
15min

Phase 3 Plenum

Share

- Share your results and experience with the group

04

Documentation

AI As An Aid To Documentation



Use AI to create and keep documentation up to date



Doku-
mentation

- Arc42 documentation
- Architecture diagrams
- Environment-specific configurations
- README.md
- AGENTS.md (check and shorten manually)
- Instruction files (check and shorten manually)

Text Based File Formats

AI is very good at creating text-based files:

- Markdown
- embedded diagrams
 - Mermaid
 - PlantUML
 - ASCII art
 - asciidoc (formatted text, can replace Word)
- Display in
 - GitHub / GitLab
 - Confluence (must be rendered as an image)



Plugins for VSCode



Mermaid

Mermaid Chart [mermaidchart.com](#) | 421,371 | ★★★★★ (23)

The official Mermaid Editor plugin by the Mermaid open source team, now with AI-powered diagramming! Create, edit and preview diagrams seamlessly within VS Code

Set Color Theme Disable Uninstall ☒ Auto Update

DETAILSFEATURESCHANGELOG

Mermaid Chart extension for Visual Studio Code

The official Mermaid extension for Visual Studio Code enables developers to seamlessly create, edit, preview and integrate mermaid diagrams from within the VS Code.



AsciiDoc

asciidocor [asciidocor.org](#) | 740,784 | ★★★★★ (10)

Provides rich language support for AsciiDoc.

Disable Uninstall ☒ Auto Update

DETAILSFEATURES

AsciiDoc support for Visual Studio Code

VS Marketplace v4.2.2 installs 740.55K rating 4.4/5 (10) zulip join chat

An extension that provides live preview, syntax highlighting, and snippets for the AsciiDoc format using AsciiDoctor.

Installation

Identifier	asciidocor.asciivscode
Version	4.2.2
Last Updated	2 weeks ago
Size	46.53MB



PlantUML

jebbs | 3,276,913 | ★★★★★ (104)

Rich PlantUML support for Visual Studio Code.

Disable Uninstall ☒ Auto Update

DETAILSFEATURESCHANGELOG

PlantUML README

Rich PlantUML support for Visual Studio Code.

build unknown

Installation

Identifier	jebbs.plantuml
Version	2.18.1
Last Updated	3 weeks ago
Size	13.10MB

Task: Documentation



Use AI to create documentation

Phase 1

Create Docs

1. Create an **overview** to help new developers get started (README.md)
2. Update the **AGENTS.md** file
3. Create a Mermaid or PlantUML **diagram of the software architecture**
4. Create a complete set of **Arc42** documentation.

Phase 2 Plenum

Share

- Display your results. Take turns, display every result
- Discuss strengths and weaknesses
- Discuss how to improve the documentation

Phase 3

Refinement

1. Enhance the documentation
2. Check if other docs (architecture etc.) must be updated

Phase 4 Plenum

Share

- Closing review with all members

Example



Doku-
mentation

I want to create architectural documentation for this application using the Arc42 template. Please check the existing documents in the @docs folder and also the @README.md file. Then examine the existing documentation in detail.

For diagrams, use Mermaid or PlantUML

05

Conclusion

For Every Story / Every Ticket

Create one after another:

1. Refined story $\leftarrow$ product / definition_ready
2. Implementation Plan $\leftarrow$ story / architecture / code_style / ...
-  3. – Implementation –
4. Review $\leftarrow$ story / plan / architecture / testing / definition_done / ...
5. If needed: – update documentation –

Prompt Dos & Don'ts

Don't:

"create a definition of done"

Also don't:

"please ..."

"can you ..."

Instead

"Help us create a Definition of Done.

What should it include?

[...]

Ask one question at a time."

What do I need to check, and what don't I need to check?

"It depends..."

How significant is my project in terms of security? For critical infrastructure: ideally, every line of code. It is a context-dependent decision that each individual or the team must make for themselves.

Generally speaking, in **rough** descending order of importance:

- "Steering Documents" (AGENTS.md, architecture.md, etc.)
- Story definitions, implementation plan, review report
- Tests
- Key areas of the code (business logic)
- Other code